SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

NAAC

PRACTICAL-6

Aim: Implement a simple RPC system using Python (XML-RPC).

PULSES:

- Remote Procedure Call (RPC)
- Client–Server model
- XML-RPC protocol
- Python xmlrpc library
- Create an RPC server
- Register functions on the server
- Start the server and listen for requests
- Create an RPC client
- Client calls remote functions
- Server executes and returns result
- Client sends a function request to server
- Server executes the function
- Result is sent back to client
- Client receives output transparently
-

Program:

Step 1: Create RPC server

Save this as server.py

```
from xmlrpc.server import SimpleXMLRPCServer

server = SimpleXMLRPCServer(("localhost", 8000))

#Define functions

def add(x,y):
    return x+y

def subtract(x,y):
    return x-y

def multiply(x,y):
    return x*y

def divide(x,y):
    if y==0:
        return "Error: Division by zero!"
```

```
return x+y

# Register functions
server.register_function(add,"add")
server.register_function(subtract)
server.register_function(multiply)
server.register_function(divide),

# Run server
server.server_forever()
```

- Step 2: Create RPC client

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy("http://localhost")
Print(proxy.add(10,5))
Print(proxy.subtract(10,5))
Print(proxy.multiply(10,5))
Print(proxy.divide(10,5))
```

Output:

Addition : 15
Subtraction : 5
multiplaction : 50
Division : 2.0

MCQ:

1. RPC stands for:
A) Remote Program Control B) Remote Procedure Call
C) Random Process Communication D) Remote Process Connection

2. The main advantage of RPC is that it:
A) Uses shared memory B) Hides network communication details
C) Works only on the same machine D) Is faster than all IPC methods

3. XML-RPC uses which protocol for communication?
A) FTP B) TCP only C) HTTP D) UDP

4. Which Python class is used to create an XML-RPC server?
A) XMLServer B) RPCServer C) SimpleXMLRPCServer D) ServerSocket

Resolution: The simple RPC system using python was successfully implemented the server was created using the SimpleXMLRPCSever class and multiple functions were registered.

Signature with Date of Completion	
Marks out of 10	

SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

PRACTICAL-7

Aim: Client remotely invokes procedures on server.

PULSES:

- Client program calls a function that is executed on a remote server, as if it were a local function call.
- The client sends a request (procedure name + parameters) to the server
- The server executes the procedure
- The result is sent back to the client
- The client does not need to know where or how the procedure is executed
- This communication happens transparently using network protocols.

Program: Step 1:

```
From xmlrpc.server import SimpleXMLRPCServer
server = SimpleXMLRPCServer(("8000"))
def square(n):
    return n*n
def cube(n):
    return "Hello "+name
server.register_functions(square)
server.register_functions(cube)
server.register_functions(greet)
server.serve_forever()
```

Step-2

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy
Print(proxy.square(4))
```

Output:

Square : 16

cube : 27

Hello student

MCQ:

1. Client remotely invokes procedures on server refers to:

A) Message Passing B) Shared Memory C) Remote Procedure Call D) Pipelining

2. In RPC, where does the procedure actually execute?

A) Client machine B) Both client and server C) Server machine D) Kernel only

3. Which architecture is used in RPC?

A) Peer-to-Peer B) Client–Server C) Master–Slave D) Distributed Memory

Resolution:

The client successfully invoked remote procedures on the server using XML-RPC. The server executed the requested functions and returned the result to the client over the network.

Signature with Date of Completion	
Marks out of 10	

SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

PRACTICAL-8

Aim: Simulate Cristian's and Berkeley algorithms using Python.

PULSES:

- Used in client–server systems
- One machine acts as a time server
- Client requests time and adjusts its clock considering network delay

Program:

1. Cristian Algorithm.

```
import time
import random

def server_time():
    return time.time()

def cristian_algorithm():
    t0 = time.time()
    delay = random.uniform(0.1, 0.5)
    time.sleep(delay)
    server_clock = server_time()
    t1 = time.time()
    round_trip_delay = t1 - t0
    adjusted_time = server_clock + (round_trip_delay/2)
    print(server_clock)
    print(adjusted_time)
```

Cristian_algorithm()

2 Berkeley Algorithm

```
import time
import random
clocks = [
    time.time() + random.randint(-5,5),
    time.time() + random.randint(-5,5)
    time.time() + random.randint(-5,5)
]
Print("Initial clock Times:")
for i, clock in enumerate(clocks):
average_time = sum(clocks) / len(clocks)
Print("\nAverage Time:", average_time)
```

```
# Adjust clocks
Print("\n Adjusted clock Times :")
For i in range(len(clocks)):
        clocks[i] = average_time
        print("{}".format(clocks[i]))
```

Output: Cristian:-

client requesting time from server ...
server Time : 1710000000000.45
Adjusted client Time : 1710000000000.07

Berkeley:

Initial clock
machine 1: ...
machine 2: ...
machine 3: ...

Average Time: ...
Adjusted clock Time ...
machine 1: ...
machine 2: ...
machine 3: ...

MCQ:

1. Cristian's algorithm is based on which architecture?

A) Master-Slave (B) Client-Server C) Peer-to-Peer D) Distributed Hash

2. In Cristian's algorithm, the client adjusts its clock using:

A) Server time only (B) Round Trip Time (RTT) C) Average of all nodes D) Random offset

3. Berkeley algorithm is mainly used in:

A) Centralized systems (B) Distributed systems C) Single-user systems D) Real-time OS only

Resolution: The cristian's algorithm and Berkeley algorithm were successfully simulated using Python. Clock synchronization was achived by adjusting client time based on server response in Cristian's method and by averaging clock values in Berkeley method.

Signature with Date of Completion	
Marks out of 10	

SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

PRACTICAL-9

Aim: Compare system time offsets across nodes.

PULSES:

- Clock synchronization
- System time offsets
- Distributed systems
- Time comparison across nodes
- Assume multiple nodes with different local times
- Select a reference time (master/average time)
- Calculate offset = reference time − node time
- Compare offsets of all nodes
- Identify nodes that are ahead or behind
- Time offset indicates clock drift
- Positive offset → node clock is slow
- Negative offset → node clock is fast
- Zero offset → clock is synchronized

Program:

```
import time
import random

nodes = {
    "node 1": time.time() + random.randint(-5, 5
    "node 2": time.time() + random.randint(-5, 5
    "node 3": time.time() + random.randint(-5, 5
    "node 4": time.time() + random.randint(-5, 5
}
print("Local Times of nodes:")
for node, t in nodes.items():
    print(f"{node}: {t}")
```

```
reference_time = sum(nodes.values()) / len(nodes)
print("\n Reference time (Average Time: ", reference
                                        _time)

print("\n Time offsets:")
for node, t in nodes.items():
    offset = reference_time - t
if offset > 0:
    print("--> clock is slow")
elif offset < 0:
    print("--> clock is Fast")
else:
    print("--> clock is synchronised")
print().
```

Output: local times of nodes:

Node 1: ....
Node 2: ....
Node 3: ....
Node 4: ....
Reference time: ....
node 1 offset: ...
→ clock is slow.

MCQ:

1. System time offset is the difference between:
A) Two server clocks B) Node time and reference time
C) Execution time and wait time D) System time and CPU time

2. A positive time offset indicates that the node clock is:
A) Fast B) Accurate C) Slow D) Faulty

3. Which algorithm uses average time to compute offsets?
A) Cristian's B) Lamport's C) Berkeley D) NTP

Resolution: The system time offset across multiple nodes were successfully compared using Python. The reference time was calculated using the average of all node time, and offsets were computed to identify whether each node clock was fast, slow or synchronized.

Signature with Date of Completion	
Marks out of 10	

PRACTICAL-10

Aim: Implement Bully or Ring Algorithm using Python processes.

PULSES:

- Each process has a unique ID
- When a process detects coordinator failure:
- It sends ELECTION message to higher-ID processes
- If no higher process responds:
- It becomes the new coordinator
- Highest-ID alive process always wins (bully behavior)

Program:

```
import random

processes = [1, 2, 3, 4, 5]

Coordinater = 5

Print (Coordinater)

processes.remove (Coordinater)

print ("Coordinater Failed !")

Initiater = 2

Print ("In process", initiator)

higer_processes = [p for p in processes if
                                p > initiater]

if higer_processes:
  new_coordinator = max (higher_process
```

```
else:
    new_coordinator = initiator

print("\n new coordinator is :", new_coordinator)
```

Output: Initial coordinator = 5
Coordinator failed !

Process 2 starts election
Election message sent to : [3, 4]
new coordinator is : 4

MCQ:

1. The Bully Algorithm is used for:

A) Deadlock prevention B) Leader election C) Memory management D)Synchronization

2. In Bully Algorithm, which process becomes coordinator?

A) Lowest ID B) Random ID C) Highest active ID D) First responder

3. Bully Algorithm assumes that:

A) Processes are anonymous B) Processes have unique IDs

C) Network is unreliable D) Only two processes exist

Resolution: The bully and ring election algorithms were successfully implemented using Python. Thus, leader election in distributed system was simulated successfully.

Signature with Date of Completion	
Marks out of 10	

SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

NAAC

PRACTICAL-11

AIM: Implement Round Robin or Least Load scheduling in Python.

PULSES:

- Define processes with burst times
- Select a fixed time quantum
- Execute each process for one time quantum
- If remaining burst time exists, move process to the back of queue
- Repeat until all processes finish
- Each process gets CPU for a fixed time slice
- Preemptive scheduling
- Ensures fairness
- Reduces starvation

Program:

```
def round_robin(processes, burst_time, time_quantum):
    n = len(processes)
remaining_time = burst_time.copy()
    time = 0

while true:
        done = True

    for i in range(n):
            if remaining_time[i] > 0
                done = False

if remaining_time[i] > time_quantum:
        time += time_quantum
    reamining_time[i] -= time_quantum

else:
```

```
    time += remaining_time[i]
    Printf"{processes[i]}
    remaing_time[i] = 0
    if done:
        break
Print("\n Total time", time)
```

Input

```
processes = ["P1", "P2", "P3"]
burst_time = [10, 5, 8]
time_quantum = 3
```

Output:-

P1 executed for 3 units
P2 executed for 3 units
P3 executed for 3 units
P1 executed for 3 units
P2 executed for 2 units
P3 executed for 3 units
P1 executed for 3 units
P3 executed for 2 units
P1 executed for 1 units

MCQ:

1. Round Robin scheduling is a:
A) Non-preemptive scheduling B) Preemptive scheduling C) Priority scheduling D) Batch scheduling

2. The time slice used in Round Robin scheduling is called:
A) Turnaround time B) Burst time C) Time quantum D) Waiting time

3. Round Robin scheduling is mainly used in:
A) Batch systems B) Real-time systems C) Time-sharing systems D) Distributed file systems

Resolution: Round Robin scheduling was successfully implemented using Python. Each process was executed for a fixed time quantum in a cyclic manner until completion.

Signature with Date of Completion	
Marks out of 10	

SILVER OAK UNIVERSITY

PRACTICAL-12

AIM: Implement Ricart–Agrawala Algorithm using message passing.

TOPICS:

- Ricart–Agrawala Algorithm
- Distributed Mutual Exclusion
- Message Passing
- Logical (Lamport) Timestamps
- Python multiprocessing & Queue
- Create multiple processes (nodes)
- Each process maintains a logical clock
- When a process wants to enter the critical section:
- Sends REQUEST(timestamp, pid) to all other processes
- Receiving process replies OK if it is not interested or has lower priority
- Requesting process enters Critical Section (CS) after receiving all OKs
- On exit, deferred replies are sent

Program:

```
import multiprocessing
import time
import random

def process(pid, total_process, request_queue):
    logical_clock = 0
    time.sleep(random.randint(1,3))
    logical_clock += 1
    timestamp = logical_clock

for i in range(total_processes):
    if i != pid:
        request_queue.put((pid, request, timestamp))

replies = 0
```

```
while replies < total_processes - 1:
    msg = request_queue.get()
    if msg[0] == "ok" and msg[3] == pid:
        replies += 1
print(f"process {pid} entering critical section")
time.sleep(2)
print(f"process {pid} exiting critical section")

for i in range(total_processes)
    if i != pid:
        request_queue.put(("ok", logical_clock, pid,i))
if __name__ == "__main__":
    total_processes = 3
request_queue = multiprocessing.Queue()
processes = []
for pid in range(total_processes):
    p = multiprocessing.Process(target=process,
                                args=(pid, total_processes,
                                request_queue))
    processes.append(p)
    p.start()
for p in processes:
    p.join()
```

Output:-

Process 0 sending REQUEST at time 1
Process 1 sending REQUEST at time 1
Process 2 sending REQUEST at time 1
Process 0 entering Critical Section
Process 0 exiting Critical Section
Process 1 entering Critical Section.

MCQ:

1. Ricart–Agrawala algorithm is used for:
A) Leader election B) Deadlock detection C) Distributed mutual exclusion D) Clock synchronization

2. Ricart–Agrawala algorithm uses which timestamps?
A) Vector clocks B) Physical clocks C) Lamport logical clocks D) System clocks

3. How many replies are required before entering CS?
A) One B) Half of processes C) All other processes D) Only coordinator

Resolution: The Ricart - Agrawala algorithm was successfully implemented using python multiprocessing and message Passing

Signature with Date of Completion	
Marks out of 10	

SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

PRACTICAL-13

AIM: Configure and explore NFS or HDFS concepts (local simulation).

PULSES:

- Network File System (NFS)
- Distributed file system
- Client–Server architecture
- Remote file access

Program:

Simple HDFS concept simulation

```
file_data = "This is a sample file stored in distributed
            blocks!"
block_size = 10
blocks = [file_data[i:i+block_size] for i in range(0, len
                                    (file_data), block_size)]
Print("File divided into blocks:\n")
for i, block in enumerate(blocks):
    Print(f"Block {i+1}: {block}")
```

Output:-

File divided into blocks:
Block 1: This is a
Block 2: sample file
Block 3: e stored i
Block 4: n distribu
Block 5: ted blocks
Block 6: .

MCQ:

1. NFS stands for
A) Network File Sharing B) Network File System C) Node File System D) New File Service

2. NFS follows which architecture?
A) Peer-to-peer B) Client–Server C) Master–Slave D) Standalone

3. Which command is used to export shared directories in NFS?
A) mount B) showmount C) exportfs D) chmod

Resolution: The ~~NFS~~ HDFS concepts were successfully explored using python simulation. Remote file access was demonstrated using a client-server model, and file block distribution was simulated for HDFS.

Signature with Date of Completion	
Marks out of 10	

SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

NAAC

PRACTICAL-14

AIM: Simulate wait-for graph using Python.

PULSES:

- A Wait-For Graph (WFG) is used in deadlock detection
- Nodes → Processes
- Edge Pi → Pj means Pi is waiting for a resource held by Pj
- Cycle in graph ⇒ Deadlock

Program:

```
from collections import defaultdict

class waitforgraph:
    def __init__(self):
        self.graph = defaultdict(list)

def add_edge(self, process1, process2):
    self.graph[process1].append(process2)

def add_edge(self, processes):
    visited = set()
    rec_stack = set()

def dfs(node):
    visited.add(node)
    rec_stack.add(node)
for neighbor in self.graph[node]:
    if neighbor not in visited:
        if dfs(neighbor):
            return true
```

```
        rec_stake.remove(node)
        return False
    for process in list(self.graph):
        if process not in visited:
            if dfs(process):
                return True
    return False

wfg = WaitForGraph()
n = int(input("Enter number of edges (waiting relations): "))
print("Enter number in format: Pi Pj")
for _ in range(n):
    p1, p2 = input().split()
    wfg.add_edge(p1, p2)

print("\n Wait-for Graph:")
for process in wfg.graph:
    print(process, "->", wfg.graph[process])
if wfg.detect_deadlock():
    print("\n Deadlock Detected")
else:
    print("\n No Deadlock")
```

SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

Output:

wait-for Graph
P1 -> ['P2']
P2 -> ['P3']
P3 -> ['P1']

MCQ:

1. A wait-for graph is mainly used for
A) Process scheduling B) Deadlock prevention (C) Deadlock detection D) Memory allocation

2. In a wait-for graph, nodes represent
A) Resources B) Files C) Processes (D) CPUs

3. An edge Pi → Pj in a wait-for graph means
A) Pi is executing Pj
B) Pi holds a resource of Pj
(C) Pi is waiting for a resource held by Pj
D) Pj is waiting for Pi

Resolution: The wait-for Graph was successfully simulated using Python. processes were represented as nodes. If no cycle was found, the system was considered deadlock-free.

Signature with Date of Completion	
Marks out of 10	